Assignment 2

Q1. Compare Relational Databases (RDBMS) and NoSQL Databases with respect to schema, scalability, data model, consistency, performance, and applications. Give suitable examples.

Answer:

A **Relational Database (RDBMS)** stores data in tables made of rows and columns and is queried with SQL. A **NoSQL (Not Only SQL)** database stores data in non-tabular models (key-value, document, column-family or graph) and is designed for large-scale, distributed and rapidly changing data.

Basis	RDBMS	NoSQL
Schema	Fixed, predefined schema; every row has the same columns. Changing it needs ALTER TABLE.	Dynamic / schema-less (or flexible); each record can have different fields.
Data model	Tables with rows and columns; relations through primary and foreign keys; normalized data.	Key-value, document, column-family or graph; aggregates and denormalized data.
Scalability	Mainly **vertical** (bigger server); horizontal scaling is hard and complex.	Mainly **horizontal** (add more commodity servers) using sharding and replication.
Consistency	Strong consistency with full **ACID** transactions.	Often **BASE** / eventual consistency with tunable levels; ACID mostly at single-record level (some support multi-document).
Performance	Excellent for complex joins and transactions on moderate data; slows down on huge, distributed data.	Very fast reads/writes on huge data because of no joins, denormalization, and distributed processing.
Query language	SQL (standard).	Database specific: MQL, CQL, Cypher, simple get/put APIs.
Applications	Banking, accounting, ERP, payroll, reservation systems.	Social media, IoT, real-time analytics, content management, catalogs, recommendation engines.
Examples	MySQL, Oracle, PostgreSQL, SQL Server.	MongoDB, Cassandra, HBase, Redis, Neo4j.

Q2. Explain the following NoSQL databases and their major use cases: a) MongoDB b) Apache Cassandra c) Apache HBase d) Neo4j

Answer:

a) MongoDB (Document database)

MongoDB stores data as JSON-like **BSON documents** grouped into **collections**. Documents in one collection can have different fields, and can contain nested documents and arrays. It supports a rich query language, indexing, an aggregation framework, **replica sets** for availability and **sharding** for horizontal scaling.

Use cases: content management systems, product catalogs, user profiles, mobile and web back-ends, real-time analytics, blogging and e-commerce platforms.

b) Apache Cassandra (Wide-column database)

Cassandra is a distributed **column-family** database with a **peer-to-peer (masterless) ring** architecture. Data is partitioned by a partition key and replicated to several nodes. It offers very high write throughput, linear scalability, tunable consistency (ONE, QUORUM, ALL) and no single point of failure. It is queried using CQL, which looks like SQL.

Use cases: IoT and sensor data, time-series data, event and log storage, messaging systems, activity feeds, always-on applications that need multi-data-centre availability.

c) Apache HBase (Column-oriented database on Hadoop)

HBase is modelled on Google's BigTable and runs on top of **HDFS**. Data is stored in tables with a row key and column families; tables are split into **regions** served by RegionServers. It gives strongly consistent, random, real-time read/write access to very large datasets and integrates with Hadoop and MapReduce.

Use cases: large-scale messaging (e.g. Facebook messages), web indexing, time-series and metrics, analytics on huge sparse tables, counters.

d) Neo4j (Graph database)

Neo4j stores data as **nodes** (entities), **relationships** (edges) and **properties**. Relationships are stored as direct pointers, so traversals are fast without joins. It uses the **Cypher** query language and supports full ACID transactions.

Use cases: social networks (friends, followers), recommendation engines, fraud detection, knowledge graphs, network and IT dependency mapping, route finding.

Q3. Explain the RDBMS approach and discuss the major challenges faced by traditional relational databases when handling large-scale, distributed, and rapidly changing data.

Answer:

RDBMS approach: Data is organized into **tables (relations)**. A schema is defined first, data is **normalized** to remove redundancy, tables are linked by primary and foreign keys, and SQL is used to define, query and manipulate data. Reliability is ensured through **ACID** properties (Atomicity, Consistency, Isolation, Durability) and joins combine data from many tables. This approach works very well for structured data with clear relationships, such as banking and accounting.

Major challenges with large-scale, distributed and changing data

- **Scalability limits:** RDBMS scales mostly vertically (a more powerful, costly server). Scaling out across many machines is difficult because tables and joins are tightly connected.
- **Expensive joins:** Normalized data needs many joins; on huge distributed tables joins become very slow and network heavy.
- **Rigid schema:** The schema must be fixed in advance. Frequent changes in data structure require ALTER TABLE, downtime and migration, which is hard for rapidly changing requirements.
- **Unstructured / semi-structured data:** Documents, JSON, logs, images and sensor data do not fit naturally into rows and columns.
- **Object-relational impedance mismatch:** Application objects (nested, hierarchical) must be broken into several tables and re-assembled.
- **Distributed transactions:** Keeping ACID guarantees across many servers needs two-phase commit and locking, which reduces performance and availability.
- **High write throughput and latency:** Handling millions of writes per second (IoT, social feeds) is difficult with locking, logging and index maintenance.
- **Availability and cost:** Sharding done manually by the application is complex; high-end hardware and licences are expensive; failures of a single primary can stop the service.

Q4. What is the NoSQL approach? Explain how NoSQL databases address the scalability, flexibility, and performance limitations of traditional RDBMS.

Answer:

NoSQL (Not Only SQL) is an approach to database design that uses non-relational data models and is built for distributed storage on clusters of cheap commodity servers. Instead of strict tables and joins, it stores data as aggregates (key-value pairs, documents, column families or graphs) and trades some strict consistency for scalability, availability and speed. It follows the **CAP theorem** (a distributed system can guarantee only two of Consistency, Availability and Partition tolerance at the same time) and the **BASE** model (Basically Available, Soft state, Eventually consistent).

RDBMS limitation	How NoSQL addresses it
Scalability: vertical scaling only, costly	**Horizontal scaling** - data is automatically split by **sharding** across many servers; new nodes are added without downtime; **replication** gives fault tolerance.

RDBMS limitation	How NoSQL addresses it
Flexibility: fixed schema, hard to change	**Schema-less / dynamic schema** - new fields can be added anytime; different records may have different structures; supports JSON, graphs, key-value, wide columns.
Performance: joins and locking slow down big data	**Denormalized aggregates** are read in one lookup (no joins); lightweight consistency, in-memory caches, log-structured storage and parallel processing give very fast reads and writes.
Availability: single master / single point of failure	**Replication** and masterless designs (e.g. Cassandra) keep the system running when nodes fail.
Cost: expensive hardware and licences	Runs on **commodity hardware**; most NoSQL products are open source.

Q5. Explain the following NoSQL data models with suitable examples: a) Key-Value b) Document c) Column-Family d) Graph

Answer:

a) Key-Value Data Model

The simplest model: data is stored as a **(key, value)** pair. The key is unique and the value is opaque (string, number, JSON, binary). Reads and writes are done only by key, so it is extremely fast and easy to scale. **Examples:** Redis, Riak, Amazon DynamoDB. **Uses:** sessions, caching, shopping carts, user preferences.

```
SET student:101 "Amit Sharma"
GET student:101        -> "Amit Sharma"
```

b) Document Data Model

Data is stored as self-describing **documents** (JSON/BSON/XML) inside collections. A document may contain nested objects and arrays, and different documents may have different fields. Queries can be made on any field. **Examples:** MongoDB, CouchDB. **Uses:** content management, catalogs, user profiles.

```
{ "_id": 101, "name": "Amit Sharma", "course": "BCA",
  "skills": ["Python", "MongoDB"],
  "address": { "city": "Meerut", "state": "UP" } }
```

c) Column-Family Data Model

Data is stored in rows identified by a **row key**, and columns are grouped into **column families**. Each row can have a different set of columns (sparse), and data of a column family is stored together on disk, making it efficient for large datasets and write-heavy workloads. **Examples:** Cassandra, HBase. **Uses:** time-series, logs, IoT, messaging.

Row key	personal:name	personal:city	academic:course	academic:marks
101	Amit Sharma	Meerut	BCA	78
102	Neha Verma	-	BCA	85
103	Rohit Singh	Noida	-	69

d) Graph Data Model

Data is stored as **nodes** (entities), **edges / relationships** (connections) and **properties** on both. It is best when relationships are as important as the data and queries traverse connections. **Examples:** Neo4j, Amazon Neptune. **Uses:** social networks, recommendations, fraud detection.

```
(Amit:Student)-[:ENROLLED_IN]->(DBMS:Course)<-[:ENROLLED_IN]-(Neha:Student)
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course {title:'DBMS'}) RETURN s.name
```

Q6. What are Aggregate-Oriented Databases? Explain the concept of aggregates and discuss how they are useful in NoSQL database design.

Answer:

An **aggregate** is a collection of related objects that is treated as **one unit** for storage, retrieval and updating. It is the basic unit of consistency and distribution. For example, an order together with its customer details, shipping address and order items is one aggregate. **Aggregate-oriented databases** store and retrieve data aggregate by aggregate. Key-value, document and column-family databases are aggregate-oriented, whereas relational and graph databases are not (they split data by relations and joins).

```
Order aggregate (stored as one document):
{ "order_id": 5001,
  "customer": { "name": "Amit", "city": "Meerut" },
  "items": [ { "product": "Keyboard", "qty": 1, "price": 1500 },
             { "product": "Mouse",    "qty": 2, "price": 700 } ],
  "payment": { "mode": "UPI", "status": "paid" } }
```

Usefulness in NoSQL design

- **Fast access:** The whole aggregate is read or written in one operation, with no joins.
- **Natural unit of distribution:** An aggregate always lives on one node, so sharding and replication are simple; related data is never split across servers.
- **Atomicity:** Updates inside one aggregate are atomic, giving consistency without distributed transactions.
- **Matches application objects:** Reduces the object-relational mismatch; data is stored the way the application uses it.
- **Flexibility:** Each aggregate can have its own structure.

Limitation: Queries that cut across aggregates (e.g. total sales per product across all orders) are harder and may need duplication of data or MapReduce / aggregation pipelines. The designer must choose aggregate boundaries according to how the data is accessed.

Q7. Differentiate between MongoDB's Document Model and Cassandra's Column-Family Model. Give one suitable application for each.

Answer:

Basis	MongoDB - Document Model	Cassandra - Column-Family Model
Unit of storage	BSON document in a collection	Row in a table, columns grouped by partition key and clustering columns
Structure	Nested, hierarchical documents with arrays; flexible fields	Flat wide rows; columns defined in table schema (flexible with collections)
Schema	Dynamic (optional validation)	Defined table schema (CQL), changed with ALTER
Query	Rich ad-hoc queries on any field, secondary indexes, aggregation pipeline	Queries must follow the primary key (partition key); data modelled per query
Architecture	Primary-secondary replica sets + sharding via mongos	Masterless peer-to-peer ring; all nodes equal
Consistency	Strong by default (reads from primary), tunable	Tunable per query (ONE, QUORUM, ALL), eventual by default
Strength	Flexible, developer friendly, complex queries	Very high write throughput, linear scaling, always-on availability
Suitable application	**Content management / e-commerce product catalog** where items have varying attributes	**IoT sensor data or event logging** with millions of writes per second

Q8. Explain Replication and Sharding in NoSQL databases. Why are these techniques important for distributed database systems?

Answer:

Replication means keeping **copies of the same data on multiple servers**. If one server fails, another copy can serve requests. Reads can be spread over the copies. Types: master-slave (primary-secondary) and peer-to-peer.

Sharding (horizontal partitioning) means **splitting a large dataset into smaller parts (shards)** and storing each part on a different server. Each server holds only a portion of the data, so storage and load are shared. A **shard key** decides which shard stores which record.

Basis	Replication	Sharding
Idea	Same data copied on many nodes	Different data on different nodes
Main goal	High availability, fault tolerance, read scaling	Horizontal scalability of storage and writes
Data on each node	Full copy (all data)	Only a subset (a shard)
If a node fails	Another replica takes over	That part is unavailable unless it is also replicated
Drawback	Write cost, replication lag, more storage	Complex setup, shard key choice, rebalancing

Why they are important

- **Scalability:** sharding spreads data and load so the system grows by adding servers.
- **High availability and fault tolerance:** replication keeps the data available even when servers or data centres fail.
- **Better performance:** requests are processed in parallel by several nodes; reads are balanced across replicas; data can be placed near users to reduce latency.
- **Disaster recovery and no single point of failure:** copies of data protect against data loss.
- Together they let NoSQL systems handle big data on cheap commodity hardware.

Q9. Explain the following distribution models with suitable diagrams: a) Single Server b) Sharding c) Master-Slave Replication d) Peer-to-Peer Replication

Answer:

a) Single Server

All data is stored and all requests are processed by **one machine**. It is the simplest model: no distribution, no replication, no consistency problems. **Advantage:** easy to set up and manage. **Limitation:** a single point of failure and limited by the capacity of one machine. **Use:** small applications, testing and development, graph databases for small data.

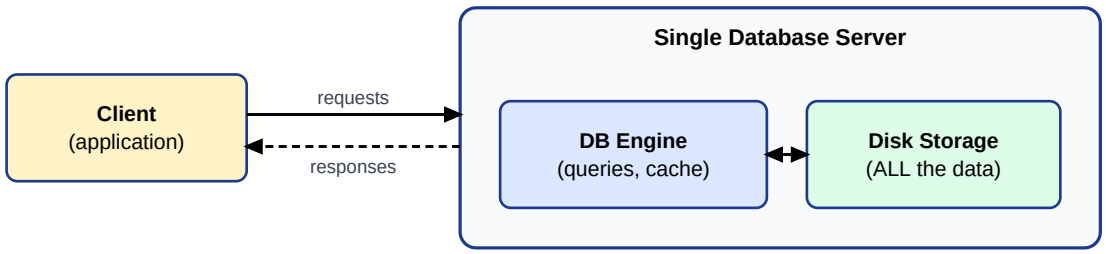

Fig 1: Single Server model

b) Sharding

The dataset is divided by a **shard key** and each part is stored on a different server. A router (or the client driver) sends each request to the correct shard. Writes and storage are distributed, so the system scales horizontally. **Advantage:** scalability and parallel processing. **Limitation:** if a shard fails its data is unavailable (unless replicated); cross-shard queries are costly.

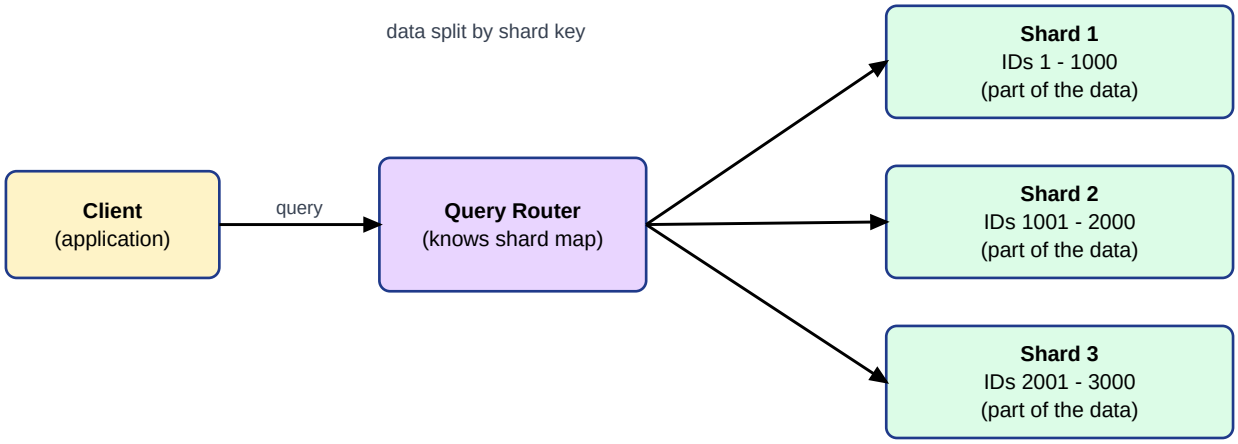

Fig 2: Sharding - data split across servers

c) Master-Slave Replication

One node is the **master (primary)** and accepts all **writes**. The changes are copied to one or more **slaves (secondaries)**, which usually serve **reads**. **Advantage:** read scaling and backup copies; simple consistency for writes. **Limitation:** the master is a write bottleneck and a single point of failure until a new master is elected; slaves may be slightly stale (replication lag).

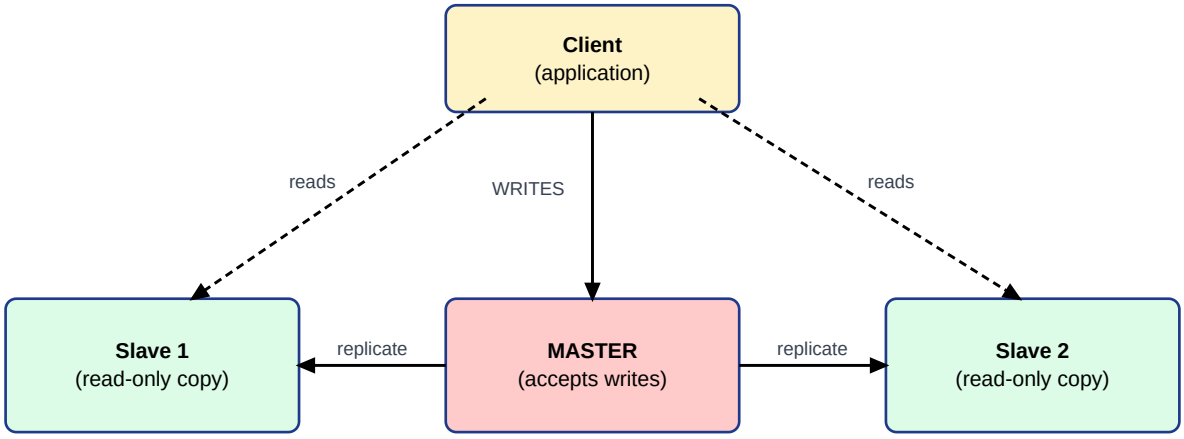

Fig 3: Master-Slave replication

d) Peer-to-Peer Replication

There is **no master**; all nodes are equal and every node can accept both reads and writes. Data is replicated among peers. **Advantage:** no single point of failure, write scaling and high availability. **Limitation:** possible inconsistency and write conflicts that need resolution (eventual consistency). Example: Cassandra.

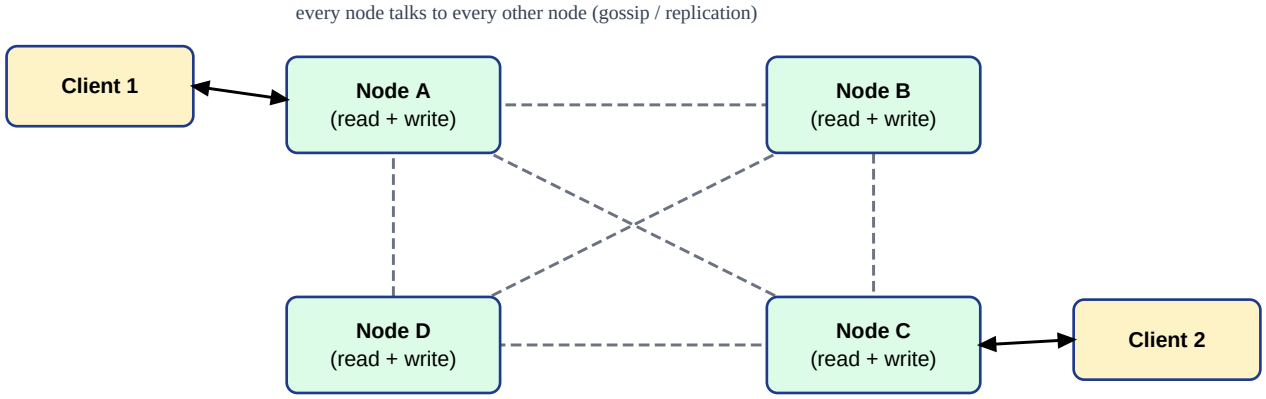

Fig 4: Peer-to-Peer replication

Q10. What is Master-Slave Replication? Explain its working, advantages, limitations, and suitable applications.

Answer:

Master-Slave (Primary-Secondary) Replication is a replication model in which one server is designated the **master** and the other servers are **slaves**. The master is the authoritative copy of the data, and the slaves keep copies of it.

Working

- All **write** operations (insert, update, delete) are sent to the **master** only.
- The master records each change in a log (e.g. oplog / binary log) and **sends the changes to the slaves**, which apply them in the same order (asynchronously or synchronously).
- **Read** operations can be served by the master or distributed among the slaves to share the load.
- **Failover:** if the master fails, a slave is promoted to master - automatically by election (MongoDB replica set) or manually by an administrator.

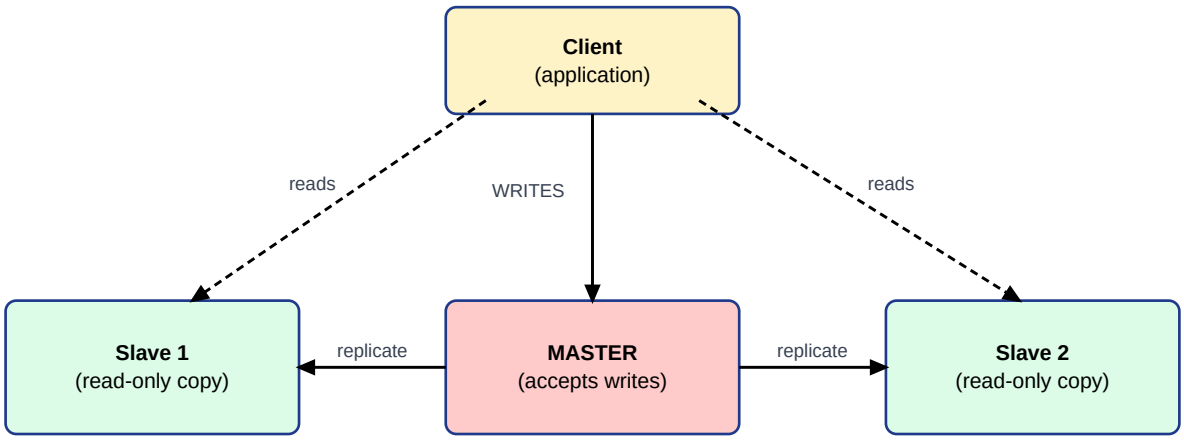

Fig: Master-Slave replication flow

Advantages

- Read scalability - many slaves can serve read requests.
- High availability and fault tolerance - slaves hold copies of data.
- Slaves can be used for backups and reporting without affecting the master.
- Simple write consistency because there is only one write point.

Limitations

- The master is a **bottleneck** and single point of failure for writes until failover completes.
- Replication lag - slaves can return stale data.
- Write scaling is not possible (all writes go to one node).
- Possible loss of recent writes if the master crashes before replicating.

Suitable applications

Read-heavy applications: e-commerce catalogs, content websites, reporting and analytics systems, banking and enterprise systems needing consistent writes and backups. Examples: MongoDB replica sets, MySQL replication, Redis replication.

Q11. Explain Peer-to-Peer Replication. How is it different from Master-Slave Replication?

Answer:

In **Peer-to-Peer replication** every node is an **equal peer**. There is no master. Any node can accept **reads and writes**, and the node that receives a write propagates it to the other replicas (using gossip and replication factor). A client can connect to any node, and the cluster continues to work when nodes fail. Consistency is controlled with consistency levels (e.g. QUORUM) and repaired by read repair, hinted handoff and anti-entropy. Example: Apache Cassandra, Riak.

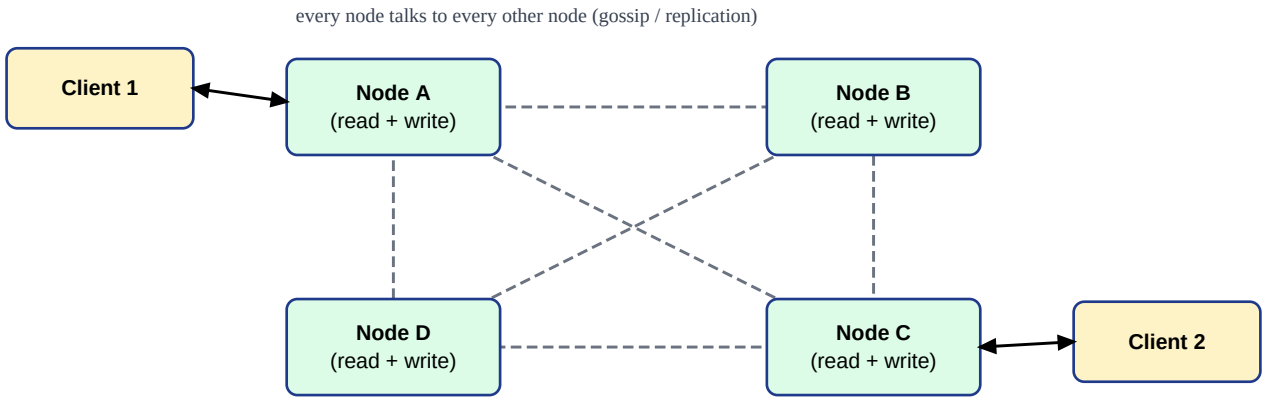

Fig: Peer-to-Peer replication (all nodes equal)

Basis	Master-Slave	Peer-to-Peer
Architecture	One master, many slaves	All nodes are equal, no master
Writes	Only on the master	On any node
Reads	Master or slaves	Any node
Single point of failure	Yes (master) until failover	No
Write scalability	Limited to one node	Scales with the number of nodes
Consistency	Easier - single write point	Eventual / tunable; conflicts possible
Failure handling	Election or manual promotion of a slave	Other nodes continue; no election needed
Examples	MongoDB, MySQL, Redis	Cassandra, Riak

Q12. What is Sharding? Explain how data is distributed across multiple servers. Discuss its advantages and challenges.

Answer:

Sharding is a technique that **horizontally partitions** a large database into smaller, independent pieces called **shards**, each stored on a different server. All shards together form the complete database.

How data is distributed

- A **shard key** (a field such as Student_ID, user_id or region) is chosen.
- **Range-based sharding:** data is split into ranges of the key (e.g. IDs 1-1000 on Shard 1, 1001-2000 on Shard 2). Good for range queries but may create hotspots.
- **Hash-based sharding:** a hash function on the key decides the shard, giving an even spread of data and load.
- **Directory / lookup based:** a lookup table keeps the mapping of keys to shards; **geo-based** sharding places data by location.
- A **router** (e.g. mongos) or the client driver, with metadata from a **config service**, sends each query to the right shard. A **balancer** moves chunks of data between shards to keep them even.

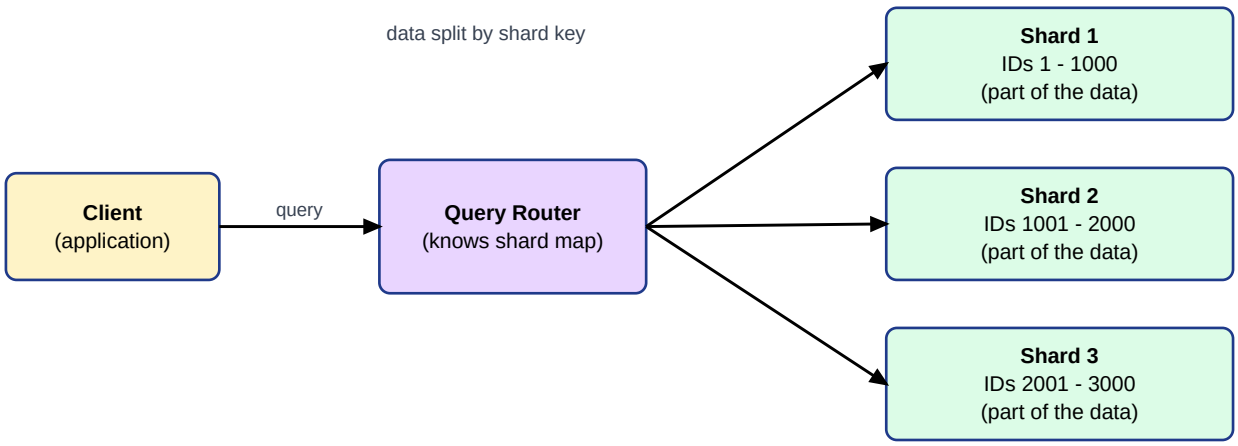

Fig: Data distributed across shards using a shard key

Advantages

- Horizontal scalability for data size and write throughput.
- Better performance through parallel processing on several servers.
- Each server handles a smaller dataset - smaller indexes, faster queries.
- Cheaper than a very large single server; capacity can grow by adding shards.
- Failure of one shard affects only part of the data (and can be limited further with replicas).

Challenges

- **Choosing the shard key** - a poor key creates hotspots and uneven data distribution.
- Complex architecture to set up, monitor and maintain.
- Queries and joins across shards are slow; multi-shard transactions are hard.
- **Rebalancing** (resharding) when adding or removing shards moves a lot of data.
- No fault tolerance by itself - a failed shard makes its data unavailable unless replicated.

Q13. Explain how Sharding and Replication can be combined in a distributed NoSQL database. Draw a suitable architecture diagram and explain its working.

Answer:

Sharding gives **scalability** but no fault tolerance, and replication gives **availability** but no write scaling. Real production systems therefore combine them: the data is first **split into shards**, and then **each shard is replicated** on several servers (a replica set). Example: a MongoDB sharded cluster.

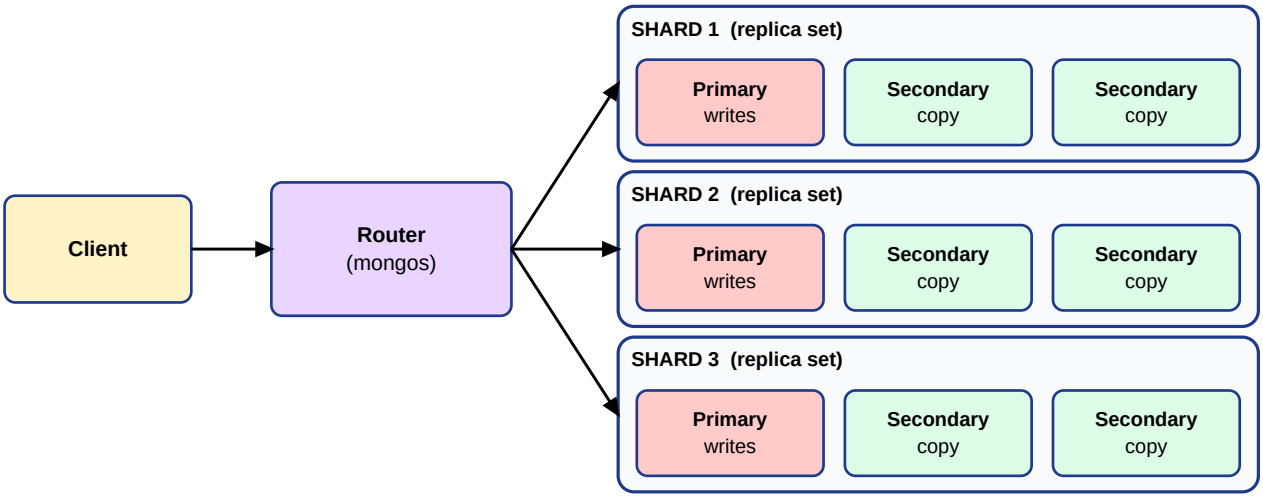

Fig: Sharding + Replication architecture (each shard is a replica set)

Components and working

- **Router (mongos):** receives the client request, uses the shard key and cluster metadata to find the right shard.
- **Config servers:** store the metadata - which range/chunk of data lives on which shard.
- **Shards:** each shard stores a part of the data and is itself a **replica set** with one **primary** and two or more **secondaries**.
- **Write path:** router -> correct shard's primary -> the primary applies the write and replicates it to its secondaries.
- **Read path:** router -> the shard's primary (or a secondary for reads that can tolerate slight lag).
- **Failure handling:** if a primary fails, the secondaries of that shard elect a new primary; the other shards are not affected.
- **Growth:** when more capacity is needed a new shard (with its own replica set) is added and the balancer moves some chunks to it.

Scalability and fault tolerance

Data and load are divided between shards, so capacity and throughput increase when more shards are added (**scalability**). Because every shard keeps multiple copies of its data on different servers, the system survives node failures without data loss and without downtime (**fault tolerance and high availability**). The cost is a more complex deployment and a larger number of servers.

Q14. What is MapReduce? Explain the Map and Reduce phases with a simple example of processing database records.

Answer:

MapReduce is a programming model for processing and generating **large datasets in parallel** across a cluster of machines. The processing is divided into two main user-defined functions, **Map** and **Reduce**, with an automatic **Shuffle and Sort** step in between. The framework takes care of distributing the work, moving data, and recovering from failures.

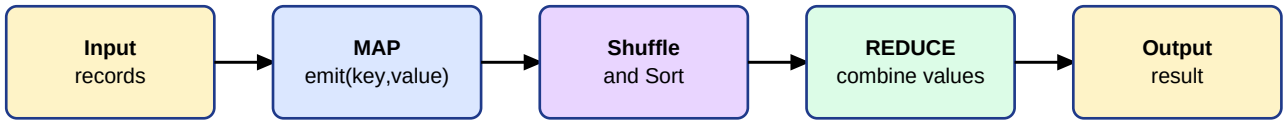

Fig: MapReduce workflow

Phase	Description
Map	Each input record is processed independently (in parallel on different nodes) and converted into **intermediate (key, value) pairs**.
Shuffle and Sort	The framework groups all intermediate values by key, so each key gets a list of its values.
Reduce	The reduce function receives a key and its list of values and **combines them** (sum, count, average, max...) into the final output.

Example: number of students and average marks in each course

Student_ID	Name	Course	Marks
1	Amit	BCA	78
2	Neha	BCA	85
3	Rohit	MCA	69
4	Priya	BBA	91
5	Karan	MCA	74
6	Isha	BCA	66

Map phase - for each record emit (Course, Marks):

```
(BCA,78) (BCA,85) (MCA,69) (BBA,91) (MCA,74) (BCA,66)
```

Shuffle and sort - group by key:

```
BCA -> [78, 85, 66]
MCA -> [69, 74]
BBA -> [91]
```

Reduce phase - count the values and compute the average:

Course	Number of students	Average marks
BCA	3	76.33
MCA	2	71.5
BBA	1	91

Implementation in MongoDB (mapReduce):

```
var map = function() { emit(this.Course, { count: 1, total: this.Marks }); };
var reduce = function(key, values) {
   var r = { count: 0, total: 0 };
   values.forEach(function(v) { r.count += v.count; r.total += v.total; });
   return r; };
var finalize = function(key, v) { v.avg = v.total / v.count; return v; };
db.students.mapReduce(map, reduce, { out: 'course_stats', finalize: finalize })
```

Q15. Explain the role of MapReduce in database processing. Discuss how it can be used for large-scale data analysis.

Answer:

Role of MapReduce in database processing

- **Parallel processing of big data:** a job is split into many small map tasks that run at the same time on different nodes, and the partial results are combined by reduce tasks. Processing time falls as nodes are added.
- **Data locality:** computation is moved to the node where the data is stored (e.g. HDFS blocks, shards), reducing network traffic.
- **Aggregation without joins:** NoSQL databases (MongoDB, HBase, Couchbase, Riak) lack SQL joins and GROUP BY in the old days; MapReduce provides grouping, counting, summing and averaging over distributed data.
- **Fault tolerance:** failed tasks are automatically re-run on other nodes, so long jobs finish reliably.
- **Simple model:** the programmer writes only map and reduce functions; distribution, scheduling and recovery are handled by the framework.

Use in large-scale data analysis

- **Log and clickstream analysis:** counting page views, errors and visitors per page or per day from terabytes of web server logs.
- **Sales and business analytics:** total sales per product, region or month; customer segmentation.
- **Text processing:** word count, building an **inverted index** for search engines.
- **Recommendation and social analysis:** finding items bought together, friends in common, PageRank style computations.
- **ETL and data cleaning:** transforming and summarizing huge datasets before loading them to a data warehouse.
- **Scientific and sensor data:** aggregating readings from IoT devices and experiments.

Typical workflow in Hadoop

```
Large data in HDFS -> split into blocks -> Map tasks run on nodes holding the blocks
-> intermediate (key,value) -> Shuffle and Sort -> Reduce tasks -> result stored in HDFS/DB
```

Limitations

- It is a **batch** model with high latency, not suited to real-time or interactive queries.
- Writing complex analytics as map and reduce functions is difficult; iterative algorithms are slow because data is written to disk between stages.
- Newer tools such as Apache Spark, Hive, and the MongoDB **aggregation pipeline** (which replaced mapReduce in MongoDB 5.0+) are faster and simpler for many tasks, but they follow the same idea of parallel map-and-combine.